

Reflection Report

This project builds a complete test suite for the ShopEasy library using various techniques. For specification-based testing, I treated the PriceCalculator as a black box and tested boundaries like zero and maximum limits using parameterized tests. Moving to structural testing, I achieved full branch coverage for the ShoppingCart. However, PIT mutation testing showed that perfect coverage does not mean bug-free code. Two mutants survived. The first mutant returned an empty list in the `getItems` method. This survived because my tests only checked if the cart was empty but did not verify the exact list contents. The second mutant returned an empty string for the `toString` method, which survived because I never explicitly tested the string output. I also added Design by Contract assertions to enforce rules directly in the code, which made the system defend itself against bad inputs.

With property-based testing, jqwik automatically generated hundreds of edge cases. During this task, a failing case occurred in the boundedness property. The calculated result was slightly larger than the maximum bound. I investigated this and found it was a standard floating-point precision error, not a logic bug, which I fixed by adding a small tolerance offset. Finally, I used Mockito to test the OrderProcessor in complete isolation. Mocking is great for testing complex logic like stopping the process when inventory fails. However, it prevents us from testing real communication between components. Mocking becomes a bad idea when we need to ensure parts work together perfectly in real life or when the external component is already simple and fast.

Looking back at the bugs revealed, the starter code was mostly solid, but the testing techniques highlighted weaknesses in the test suite itself. The mutation testing revealed gaps in assertions, showing that missing specific checks allows hidden bugs to survive. Property-based testing revealed a floating-point precision issue that manual testing missed completely. I believe specification-based testing was the most effective technique per unit of effort. It forces the tester to think about the domain and boundary values before looking at the code. It catches the most common user errors quickly and provides a strong foundation for structural tests later.

Based on the testing pyramid from Chapter 1, my test suite is heavily focused on the bottom layer. It consists almost entirely of unit tests that check individual classes like `PriceCalculator` or `ShoppingCart` in isolation. What is missing are integration tests and end-to-end tests. We mocked the external services, so we do not know if the real database or payment provider would actually work with our system. If I had one more week, I would write proper integration tests for the `OrderProcessor` using a real test database. This would ensure that the system handles real network delays and actual data persistence correctly, which unit tests simply cannot catch.